



SESSION 11

NODE.JS IN ACTION (APPLICATION DEVELOPMENT) - I

Learning Objectives

In this session, students will learn to:

- Explain the process of creating a Web application using Node.js
- Describe the steps to connect the Web application to a MongoDB database

Wonderland Widescreen is a Web-based movie application created using Node.js, HTML, and related technologies. This application will have five primary functionalities:

- Login as administrator or guest
- Add a movie
- Book seats for a movie
- Delete a movie
- Update seats for a movie

The process of complete creation and deployment of this Web application is covered across this session and the next. This session includes steps to create major part of the application using Node.js with a MongoDB database. The rest of the application creation and its deployment is covered in the next session.

11.1 Overview

In this session, a simple movie application is created using Node.js, HTML, and a MongoDB cloud database called `MovieCollectionCluster`.

Create a collection named `MovieCollection` in a database `MovieCollectionCluster`.

Figure 11.1 shows characteristics of the Wonderland Widescreen application, which are as follows:

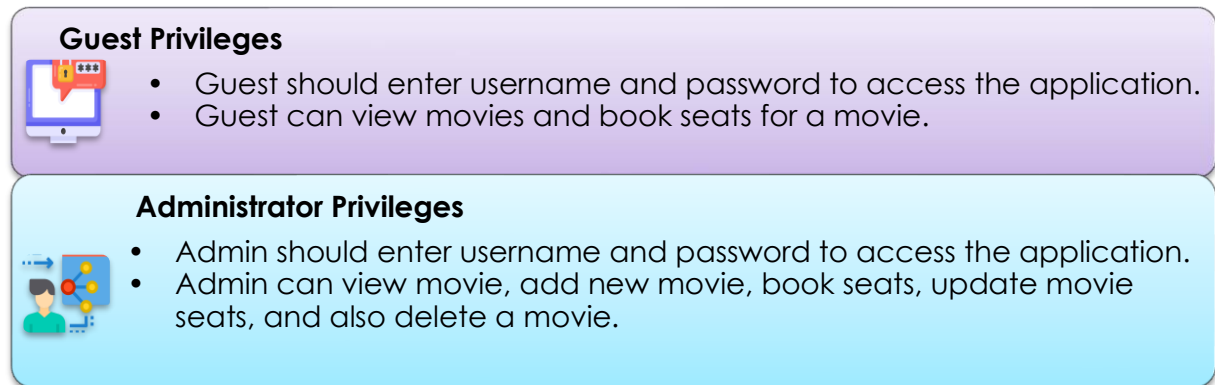


Figure 11.1: Wonderland Widescreen Application

Figure 11.2 shows the site map of the Web application with each page and its description mentioned.

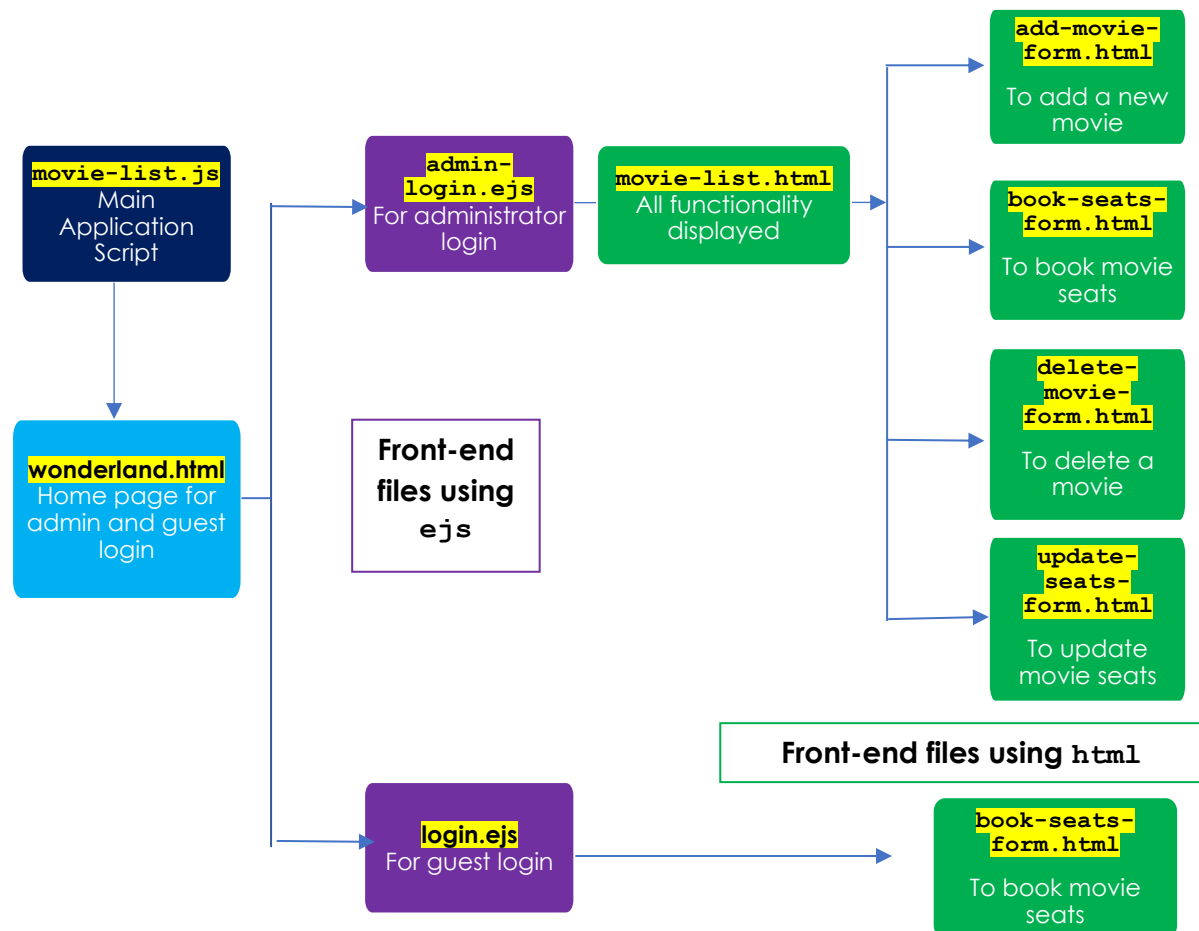
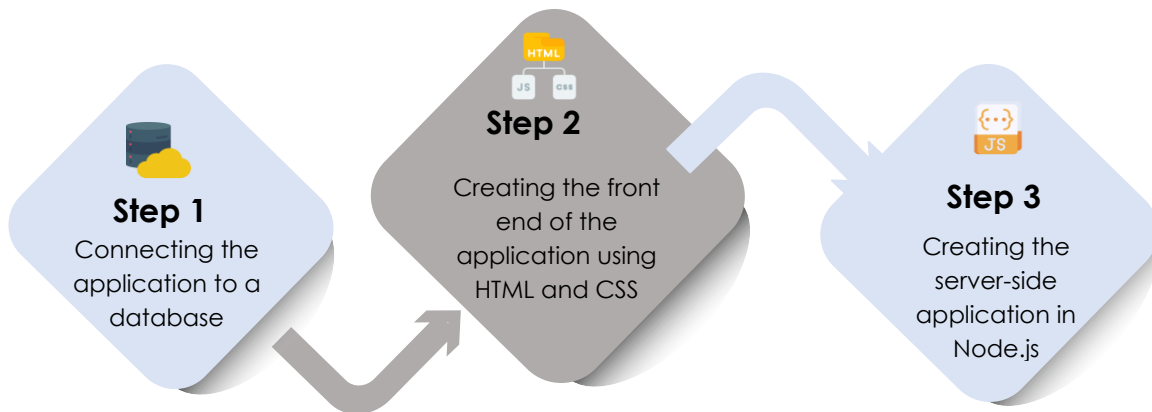


Figure 11.2: Site Map



In this session, three Web pages, `movie-list.html`, `add-movie-form.html`, and `book-seats-form.html` are created. Also, the main application entry point `movie-list.js` is created. The creation of the remaining pages such as `delete-movie-form.html` and `update-seats-form.html` is covered in the next session.

Steps to create the Wonderland Widescreen application are as follows:



11.2 Retrieving the Connection String from Mongo DB Cloud

The prerequisite to create the Wonderland Widescreen application is to deploy a database in the MongoDB cloud and retrieve the database deployment's connection string. The connection string is then used to connect the Wonderland Widescreen Node.js application with the cloud database for storing, retrieving, and processing data. Refer to Appendix A to learn how to do this.

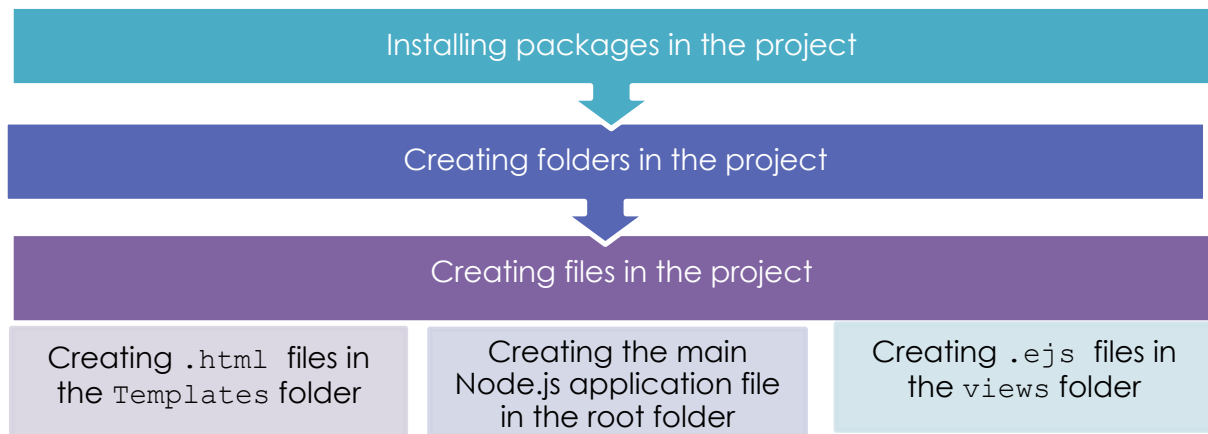
The connection string for MovieCollection appears as follows:

```
mongodb+srv://<<username>>:<<password>>@moviecollectioncluster.4x9rauo.mongodb.net/moviecollectioncluster?retryWrites=true&w=majority
```

Note that username and password placeholders in the connection string must be substituted with actual username and password provided by developer when creating the cloud database in MongoDB.

11.3 Creating the Wonderland Widescreen Application in Node.js

After the database deployment's connection string has been retrieved, one must create the Wonderland Widescreen application in Node.js which involves following steps:



11.3.1 Installing Packages in the Project

The first step is to install some important Node.js packages. One can accomplish this as follows:

1. Create a directory for the application to be developed and navigate to it:
 - a. Open Command Prompt.
 - b. Navigate to the local drive where Node.js is installed.
 - c. Navigate to the directory to create the application folder.
 - d. Create a directory named `movie-application` and change to that directory.

Figure 11.3 displays the directory path.

```
C:\Node Js>cd movie-application
```

Figure 11.3: Creation of the Application Directory

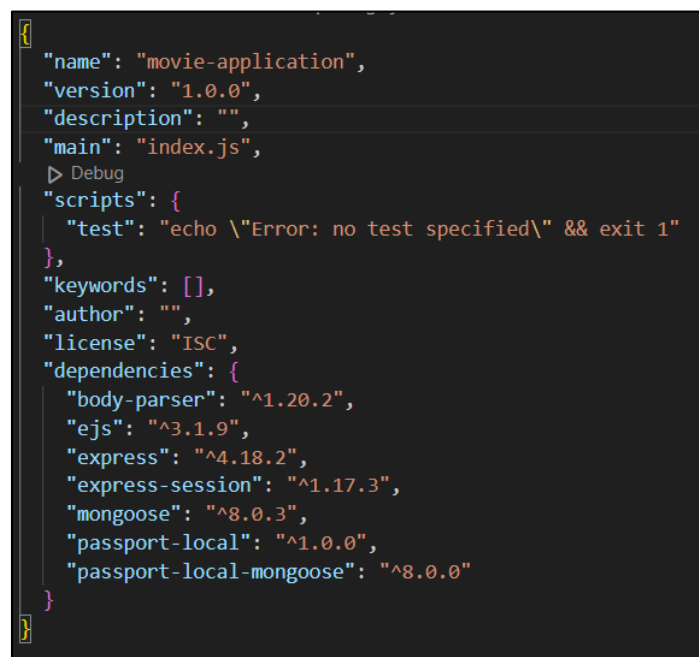
2. Create the `package.json` file to initialize a new Node.js project for the application.

3. Install the required packages such as `express`, `body-parser`, `express-session`, `passport`, and `passport-local` in the application directory using the commands:

```
npm install express
npm install body-parser
npm install express-session
npm install passport
npm install passport-local
```

Alternatively, you can install all these packages using a single `npm` command where the names of the packages are separated by spaces.

4. After installing the packages, `node_modules` folder and `package-lock.json` file are automatically created. The `package-lock.json` file includes the complete details of all the packages installed in the project. It should not be manually edited. Also, the `package.json` file is automatically updated with the version numbers of the installed packages as shown in Figure 11.4.



```
{
  "name": "movie-application",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.2",
    "ejs": "^3.1.9",
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "mongoose": "^8.0.3",
    "passport-local": "^1.0.0",
    "passport-local-mongoose": "^8.0.0"
  }
}
```

Figure 11.4: `package.json` File With Version Numbers

5. Open the `package.json` file in VS Code or Notepad. Edit the file to update the value of the `main` field to include the `engine` and `node` fields as shown in Code Snippet 1.

Code Snippet 1:

```
{
  "name": "movie-application",
  "version": "1.0.0",
  "description": "",
  "main": "movie-list.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "body-parser": "^1.20.2",
    "ejs": "^3.1.9",
    "express": "^4.18.2",
    "express-session": "^1.17.3",
    "mongoose": "^8.0.3",
    "passport-local": "^1.0.0",
    "passport-local-mongoose": "^8.0.0"
  },
  "engines": {
    "node": ">=18.17.1 <19.0.0"
  }
}
```

6. Save and close the updated `package.json` file.

11.3.2 Creating Folders in the Project

To organize the project structure, create the required folders and subfolders such as Templates and views as shown in Figure 11.5.

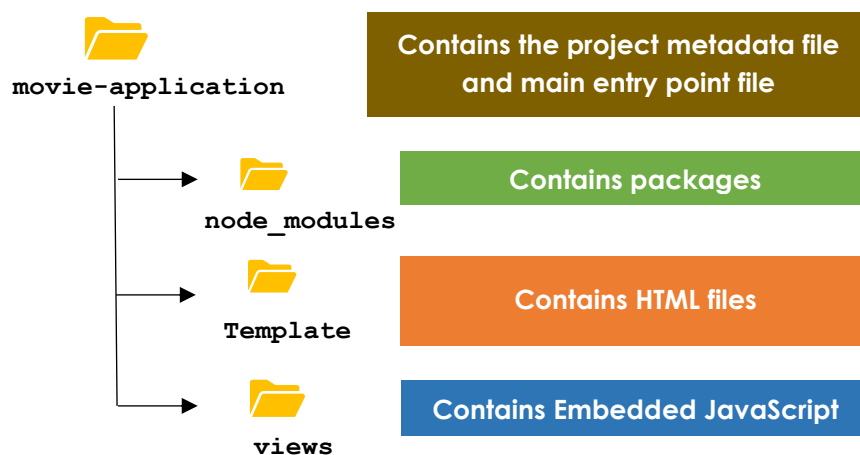


Figure 11.5: Project Folder Structure

The `node_modules` folder is created automatically when the packages are installed.

Figure 11.6 shows the listing of the `movie-application` folder.

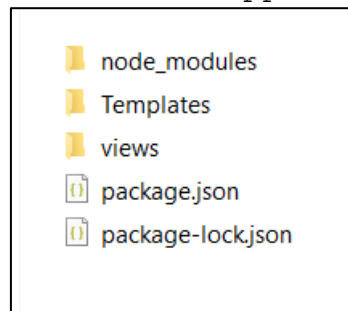


Figure 11.6: `movie-application` Folder

11.4 Creating Files in the Project

To define the user interface design, logic, and functionality of the project, create the required source code files under the specified project folders.

11.4.1 Creating `.html` files in the `Templates` Folder

The next step is to create front-end files using HTML.

➤ `movie-list.html`

Figure 11.7 shows the design for the `movie-list.html` page.



Figure 11.7: `movie-list.html`

Code Snippet 2 shows the code for movie-list.html.

Code Snippet 2:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Movie List</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <style>
    html {
      height: 100%;
    }
    body {
      text-align: center;
      background:
url("https://cdn.pixabay.com/photo/2017/01/12/17/30/warm-
and-cozy-1975215_1280.jpg");
      background-attachment: fixed;
      background-position: relative;
      background-repeat: no-repeat;
      background-size: cover;
      height: 100%;
      margin: 0;
      place-items: center;
      position: fixed;
      font-size: 20px;
    }
    .container {
      top: 30%;
      left: 50%;
      width: 50em;
      height: 18em;
      transform: translate(-50%, -50%);
      position: fixed;
      color: rgb(255, 255, 255);
      font-size: 20px;
    }
    .button {
      background-color: #000;
      border: 0.5px solid crimson;
      border-radius: 10px;
      color: #fff;
      padding: 8px;
      font-size: 20px;
      cursor: pointer; /* Pointer/hand icon */
      /* display:block; */
    }

    .movie-details-box {
```



```

margin-top: 20px;
padding: 20px;
background-color: rgba(0, 0, 0, 0.8);
color: white;
border-radius: 10px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
display: none; /* Initially hide the box */
}

.movie-category-box {
margin: 0 auto; /* Center horizontally */
margin-top: 20px;
padding: 20px;
background-color: rgba(0, 0, 0, 0.8);
color: white;
border-radius: 10px;
box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
display: none; /* Initially hide the box */
}

.button.action-button {
background-color: #8b0000;
border: 3px solid black;
cursor: pointer;
}

.button.comedy-button {
background-color: #8b0000;
border: 3px solid black;
cursor: pointer;
}

.button.drama-button {
background-color: #8b0000;
border: 3px solid black;
cursor: pointer;
}

/* Style for movie list items */
#movie-category ul {
list-style-type: none;
padding: 0;
}

#movie-category ul li {
cursor: pointer;
margin-bottom: 5px; /* Adjust as required */
/* background-color: #8fa5ac; */
}

h4 {

```

```

        color: #fcfbfb;

    }
</style>
<body>
    <div class="container">
        <h1>Welcome to the Wonderland Widescreen</h1>

        <a href="add-movie-form.html"
            ><button class="button">Add Movie</button></a
        >
        <a href="book-seats-form.html"
            ><button class="button">Book seats</button></a
        >
        <a href="delete-movie-form.html"
            ><button class="button">Delete Movie</button></a
        >
        <a href="update-seats-form.html"
            ><button class="button">Update seats</button></a
        >
        <div id="movie-list"></div>
        <br />

        <div class="button-group">
            <h4>Click each category
to view movies.&#128071;</h4>
            <button class="button action-button"
onclick="getMovies('Action')">
                Action
            </button>
            <button class="button comedy-button"
onclick="getMovies('Comedy')">
                Comedy
            </button>
            <button class="button drama-button"
onclick="getMovies('Drama')">
                Drama
            </button>
        </div>

        <!-- Create a new div for displaying movie details
-->
        <div class="movie-category-box" id="movie-
category"></div>

        <!-- Create a new div for displaying movie details
-->
        <div class="movie-details-box" id="movie-
details"></div>
        <br /><br />
    </div>

```

```

<script>
  async function getMovies(category) {
    const movieCategoryDiv =
document.getElementById("movie-category");
    try {
      const response = await fetch(`/get-
movies?category=${category}`);
      const data = await response.json();
      displayMovies(data, category);
    } catch (error) {
      console.error("Error fetching movies:", error);
    }
  }

  async function getMovieDetails(movieName) {
    try {
      const response = await fetch(`/get-movie-
details?name=${movieName}`);
      const data = await response.json();
      displayMovieDetails(data, movieName);
    } catch (error) {
      console.error("Error fetching movie details:",
error);
    }
  }

  function displayMovies(movies, category) {
    const movieCategoryDiv =
document.getElementById("movie-category");
    movieCategoryDiv.innerHTML = "";

    if (movies.length === 0) {
      movieCategoryDiv.textContent = `No movies found
for category: ${category}`;
    } else {
      const ul = document.createElement("ul");
      ul.classList.add("movie-list"); // Add a class
to the list

      movies.forEach((movie) => {
        const li = document.createElement("li");
        const movieName = movie["Movie name"];
        li.textContent = movieName;
        li.classList.add("movie-list-item"); // Add a
class to the list item

        li.addEventListener("click", () => {
          getMovieDetails(movieName);
        });
        ul.appendChild(li);
      });
    }
  }

```

```

    });
    movieCategoryDiv.appendChild(ul);
    movieCategoryDiv.style.display = "block"; //
Show the box when movies are available
  }
}

function displayMovieDetails(details, movieName) {
  const movieDetailsDiv =
document.getElementById("movie-details");
  movieDetailsDiv.innerHTML = "";

  if (details) {
    const description = details["Description"];
    const actors = details["Actors"];
    const detailsText = `
      <div><strong>Movie Name:</strong>
${movieName}</div>
      <div><strong>Description:</strong>
${description}</div>
      <div><strong>Actors:</strong>
${actors}</div>
    `;
    movieDetailsDiv.innerHTML = detailsText;

    movieDetailsDiv.style.display = "block"; //
Show the box when details are available
  }
}
</script>
</body>
</html>

```

The movie-list.html form:

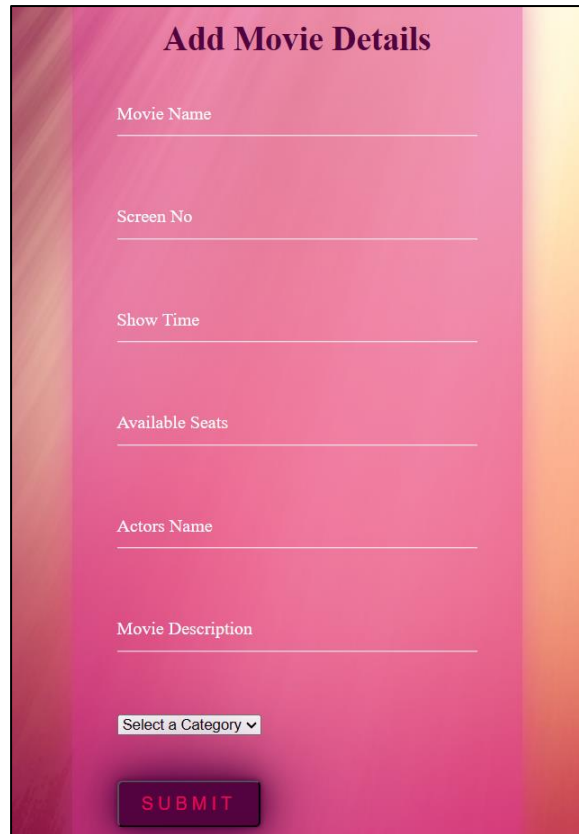
- Uses HTML and JavaScript.
- Has four button controls created using href tag to provide main functionality.
 - **Add Movie** Button: When the user clicks this button the application is redirected to the add-movie-form.html page.
 - **Book seats** Button: When the user clicks this button the application is redirected to the book-seats-form.html page.
 - **Delete Movie** Button: When the user clicks this button, the application is redirected to the delete-movie-form.html page.
 - **Update seats** Button: When the user clicks this button, the application is redirected to the update-seats-form.html page.

- Has three buttons with `onclick` events to denote movie categories. When the user clicks these buttons, available movies in that category are displayed.
 - **Action:** When this button is clicked, the `getMovies` function is called with `Action` as argument.
 - **Comedy:** When this button is clicked, the `getMovies` function is called with `Comedy` as argument.
 - **Drama:** When this button is clicked, the `getMovies` function is called with `Drama` as argument.
- Displays the **Movie names** in a list using the `movie-category` division.
- Displays the **Movie details** using the `movie-details` division.
- Defines the `getMovies(category)` function to fetch movies from the MongoDB server based on the specific category. Here, the `fetch` function is used to make request to server using `/get-movies` endpoint with the specified category.
- Defines the `getMovieDetails(movieName)` function to fetch the details of a movie from MongoDB server based on the selected movie name. This uses `/get-movie-details` endpoint.
- Defines the `displayMovies(movies, category)` function for displaying list of movies in a specified category. This function uses the `movie-category` division to display a dynamically created unordered list of movies.
- Defines the `displayMovieDetails(details, movieName)` function that uses the `movie-details` division to display details of a specific movie.

Save the code with the file name `movie-list.html` in the `Templates` folder.

➤ `add-movie-form.html`

Figure 11.8 shows the design for the `add-movie-form.html` page.



Add Movie Details

Movie Name

Screen No

Show Time

Available Seats

Actors Name

Movie Description

Select a Category

SUBMIT

Figure 11.8: `add-movie-form.html`

Code Snippet 3 shows the code for `add-movie-form.html`. The form that accepts the details of a new movie stores the movie details entered by the user to the `MovieCollectionCluster` database.

Code Snippet 3:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Add Movie Details</title>
    <style>
      html {
        height: 100%;
      }
    </style>
  </head>
  <body>
    <div>
      <h2>Add Movie Details</h2>
      <div>
        <input type="text" value="Movie Name" />
        <input type="text" value="Screen No" />
        <input type="text" value="Show Time" />
        <input type="text" value="Available Seats" />
        <input type="text" value="Actors Name" />
        <input type="text" value="Movie Description" />
        <div>
          <div>Select a Category</div>
          <div>v</div>
        </div>
        <input type="button" value="SUBMIT" />
      </div>
    </div>
  </body>
</html>
```

```

body {
    margin: 0;
    padding: 0;
    background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg");
    /* background:
url("https://cdn.pixabay.com/photo/2015/05/13/07/07/playmobil
-765110_1280.jpg"); */
    background-attachment: fixed;
    background-position: relative;
    background-repeat: no-repeat;
    background-size: cover;
}

.login-box {
    position: absolute;
    top: 50%;
    left: 50%;
    width: 400px;
    padding: 40px;
    transform: translate(-50%, -50%);
    background: rgba(226, 53, 157, 0.5);
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(226, 53, 157, 0.5);
    border-radius: 10px;
    font-size: 20px;
}

.login-box h2 {
    margin: 0 0 30px;
    padding: 0;
    color: #520340;
    text-align: center;
}

.login-box .user-box {
    position: relative;
}

.login-box .user-box input {
    width: 100%;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    margin-bottom: 30px;
    border: none;
    border-bottom: 1px solid #fff;
    outline: none;
    background: transparent;
}

```

```

.login-box .user-box label {
    position: absolute;
    top: 0;
    left: 0;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    pointer-events: none;
    transition: 0.5s;
}

.login-box .user-box input:focus ~ label,
.login-box .user-box input:valid ~ label {
    top: -20px;
    left: 0;
    color: #520340;
    font-size: 12px;
}

.login-box-button {
    position: relative;
    display: inline-block;
    padding: 10px 20px;
    color: #ee0754;
    font-size: 16px;
    text-decoration: none;
    text-transform: uppercase;
    overflow: hidden;
    transition: 0.5s;
    margin-top: 40px;
    letter-spacing: 4px;
    background: #520340;
    border-radius: 5px;
    box-shadow: 0 0 5px #520340, 0 0 25px #520340, 0 0
50px #520340,
        0 0 100px #520340;
}

.login-box-button:hover {
    background: #fff;
    color: #520340;
}
</style>
</head>
<body>
    <div class="login-box">
        <h2>Add Movie Details</h2>
        <form id="movie-form" action="/add-movie"
method="POST">
            <div class="user-box">

```



```

        <input type="text" id="movie-name" name="Movie
name" required="" />
        <label>Movie Name</label>
    </div>
    <br />
    <div class="user-box">
        <input type="text" id="screen-no" name="Screen no"
required="" />
        <label>Screen No</label>
    </div>
    <br />
    <div class="user-box">
        <input type="text" id="show-time" name="Show time"
required="" />
        <label>Show Time</label>
    </div>
    <br />
    <div class="user-box">
        <input
            type="number"
            id="available-seats"
            name="Available seats"
            required=""
        />
        <label>Available Seats</label>
    </div>
    <br />
    <div class="user-box">
        <input type="text" id="actors-names" name="Actors"
required="" />
        <label>Actors Name</label>
    </div>
    <br />
    <div class="user-box">
        <input
            type="text"
            id="movie-description"
            name="Description"
            required=""
        />
        <label>Movie Description</label>
    </div>
    <br />
    <div class="user-box">
        <!--          <input type="text" id="movie-
Category" name="Category" required="">
        <label>Movie Category</label> -->
        <select id="movie-Category" name="Category"
required="">
            <option value="" disabled selected>Select a
Category</option>

```

```
        <option value="Action">Action</option>
        <option value="Comedy">Comedy</option>
        <option value="Drama">Drama</option>
        <!-- Add more options as required -->
    </select>
    <br />
</div>
<button type="submit" class="login-box-button">
    <span></span>
    <span></span>
    <span></span>
    <span></span>
    Submit
</button>
</form>
</div>
</body>
</html>
```

The add-movie-form.html form:

- Uses HTML.
- Defines input fields for movie details such as Movie Name, Screen No, Show Time, Available Seats, Actors Name, Movie Description, and category.
- Uses the /add-movie route.

Save the code with the file name add-movie-form.html in the Templates folder.

➤ **book-seats-form.html**

Figure 11.9 shows the design for the `book-seats-form.html` page.

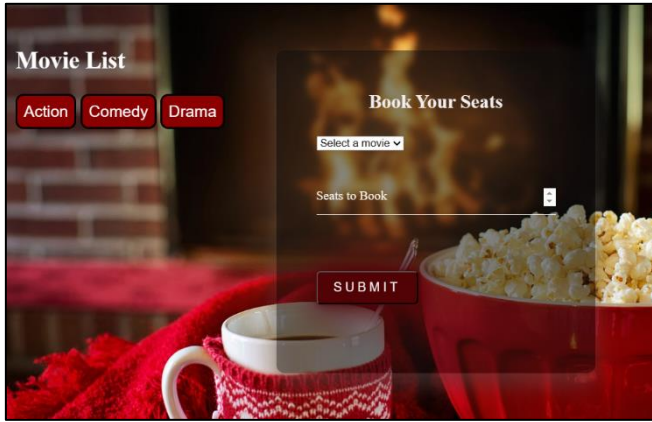


Figure 11.9: book-seats-form.html

Code Snippet 4 shows the code for `book-seats-form.html`. This form helps users to book seats for a movie.

Code Snippet 4:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Book Seats</title>
    <style>
      html {
        height: 100%;
      }
      body {
        margin: 0;
        /* padding:0;
          background:
url("https://cdn.pixabay.com/photo/2018/02/27/15/40/backgroun
d-3185765_1280.jpg"); */
        /* background:
url("https://cdn.pixabay.com/photo/2015/05/13/07/07/playmobil
-765110_1280.jpg"); */
          background:
url("https://cdn.pixabay.com/photo/2017/01/12/17/30/warm-and-
cozy-1975215_1280.jpg");
        background-attachment: fixed;
        background-position: relative;
        background-repeat: no-repeat;
        background-size: cover;
      }
    </style>
  </head>
  <body>
```

```

.login-box {
    position: absolute;
    top: 50%;
    left: 50%;
    width: 400px;
    padding: 50px;
    transform: translate(-50%, -50%);
    background: rgba(16, 15, 15, 0.5);
    box-sizing: border-box;
    box-shadow: 0 15px 25px rgba(80, 80, 80, 0.5);
    border-radius: 10px;
}

.login-box h2 {
    margin: 0 0 30px;
    padding: 0;
    color: #f3eff2;
    text-align: center;
}

.login-box .user-box {
    position: relative;
}

.login-box .user-box input {
    width: 100%;
    padding: 10px 0;
    font-size: 20px;
    color: #fff;
    margin-bottom: 30px;
    border: none;
    border-bottom: 1px solid #fff;
    outline: none;
    background: transparent;
}

.login-box .user-box label {
    position: absolute;
    top: 0;
    left: 0;
    padding: 10px 0;
    font-size: 16px;
    color: #fff;
    pointer-events: none;
    transition: 0.5s;
}

.login-box .user-box input:focus ~ label,
.login-box .user-box input:valid ~ label {
    top: -20px;
    left: 0;
}

```

```

        color: #f9f9f9;
        font-size: 15px;
    }

    .login-box-button {
        position: relative;
        display: inline-block;
        padding: 10px 20px;
        color: #f8f5f6;
        font-size: 16px;
        text-decoration: none;
        text-transform: uppercase;
        overflow: hidden;
        transition: 0.5s;
        margin-top: 40px;
        letter-spacing: 4px;
        background: #520306;
        border-radius: 5px;
        box-shadow: 0 0 5px #520306, 0 0 25px #520306, 0 0
50px #520306,
            0 0 100px #520306;
    }

    .login-box-button:hover {
        background: #fff;
        color: #520340;
    }

    /*Category button action, comedy and drama*/

    .button {
        background-color: #000;
        border: 0.5px solid crimson;
        border-radius: 10px;
        color: #fff;
        padding: 8px;
        font-size: 20px;
        cursor: pointer; /* Pointer/hand icon */
        /* display:block; */
    }

    .movie-details-box {
        margin-top: 20px;
        padding: 20px;
        background-color: rgba(0, 0, 0, 0.8);
        color: white;
        border-radius: 10px;
        box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
        display: none; /* Initially hide the box */
    }

```

```

.movie-category-box {
    margin: 0 auto; /* Center horizontally */
    margin-top: 20px;
    padding: 20px;
    background-color: rgba(0, 0, 0, 0.8);
    color: white;
    border-radius: 10px;
    box-shadow: 0 0 10px rgba(0, 0, 0, 0.5);
    display: none; /* Initially hide the box */
}

.button.action-button {
    background-color: #8b0000;
    border: 3px solid black;
    cursor: pointer;
}

.button.comedy-button {
    background-color: #8b0000;
    border: 3px solid black;
    cursor: pointer;
}

.button.drama-button {
    background-color: #8b0000;
    border: 3px solid black;
    cursor: pointer;
}

/* Style for movie list items */
#movie-category ul {
    list-style-type: none;
    padding: 0;
}

#movie-category ul li {
    cursor: pointer;
    margin-bottom: 5px; /* Adjust as required */
    /* background-color: #8fa5ac; */
}

.container {
    top: 40%; /*Display category buttons from top*/
    left: 20%;
    width: 20em; /*width of the black box*/
    height: 18em;
    transform: translate(-50%, -50%);
    position: fixed;
    color: rgb(255, 255, 255);
    font-size: 20px;
    cursor: pointer;
}

```

```

    }
    </style>
</head>
<body>
    <div class="container">
        <h2>Movie List</h2>
        <div id="movie-list"></div>

        <div class="button-group">
            <button class="button action-button"
onclick="getMovies('Action')">
                Action
            </button>
            <button class="button comedy-button"
onclick="getMovies('Comedy')">
                Comedy
            </button>
            <button class="button drama-button"
onclick="getMovies('Drama')">
                Drama
            </button>
        </div>

        <!-- Create a new div for displaying movie details -->
        <div class="movie-category-box" id="movie-
category1"></div>

        <!-- Create a new div for displaying movie details -->
        <div class="movie-details-box" id="movie-
details1"></div>
        <br /><br /><br />
    </div>

    <div class="login-box">
        <h2>Book Your Seats</h2>
        <form id="seats-form" action="/book-seats"
method="POST">
            <div class="user-box">
                <select id="movie-category" name="Movie name">
                    <option value="">Select a movie</option>
                </select>
            </div>

            <br /><br />
            <div class="user-box">
                <input
                    type="number"
                    id="seats-to-book"
                    name="seats-to-book"
                    required
                />
            </div>
        </form>
    </div>

```

```

        <label>Seats to Book</label>
    </div>
    <button type="submit" class="login-box-button">
        <span></span>
        <span></span>
        <span></span>
        <span></span>
        Submit
    </button>
</form>
<br /><br />
</div>
<script>
    async function getMovieList() {
        const movieCategoryDiv =
document.getElementById("movie-category");
        try {
            //const response = await fetch(`/get-all-
movies?category=${'Action'}`);
            const response = await fetch(`/get-all-movies`);
            const data = await response.json();
            displayMovieList(data);
        } catch (error) {
            console.error("Error fetching movies:", error);
        }
    }

    async function getMovieDetails(movieName) {
        console.log(movieName);
        try {
            const response = await fetch(`/get-movie-
details?name=${movieName}`);
            const data = await response.json();
            displayMovieDetails(data, movieName);
        } catch (error) {
            console.error("Error fetching movie details:",
error);
        }
    }

    function displayMovieDetails(details, moviename) {
        console.log(details);
        console.log(moviename);
        const movieDetailsDiv =
document.getElementById("movie-details");
        movieDetailsDiv.innerHTML = "";

        if (details) {
            const description = details["Description"];
            const actors = details["Actors"];

```



```

        const detailsText = `Description:
${description}<br>Actors: ${actors}`;
        movieDetailsDiv.innerHTML = detailsText;
    }
}

function displayMovieList(movies) {
    const movieCategoryDiv =
document.getElementById("movie-category");
    // movieCategoryDiv.innerHTML = '';

    if (movies.length === 0) {
        movieCategoryDiv.textContent = `No movies found`;
    } else {
        movies.forEach((movie) => {
            let option = document.createElement("option");
            option.setAttribute("value", movie["Movie
name"]);

            let optionText =
document.createTextNode(movie["Movie name"]);
            option.appendChild(optionText);

            movieCategoryDiv.appendChild(option);
        });

        movieCategoryDiv.style.display = "block"; // Show
//the box when movies are available
    }
}

// Category button code

async function getMovies(category) {
    const movieCategoryDiv =
document.getElementById("movie-category1");
    try {
        const response = await fetch(`/get-
movies?category=${category}`);
        const data = await response.json();
        displayMovies(data, category);
    } catch (error) {
        console.error("Error fetching movies:", error);
    }
}

async function getMovieDetails(movieName) {
    try {
        const response = await fetch(`/get-movie-
details?name=${movieName}`);
        const data = await response.json();

```

```

        displayMovieDetails1(data, movieName);
    } catch (error) {
        console.error("Error fetching movie details:",
error);
    }
}

function displayMovies(movies, category) {
    const movieCategoryDiv =
document.getElementById("movie-category1");
    movieCategoryDiv.innerHTML = "";

    if (movies.length === 0) {
        movieCategoryDiv.textContent = `No movies found for
category: ${category}`;
    } else {
        const ul = document.createElement("ul");
        ul.classList.add("movie-list"); // Add a class to
// the list

        movies.forEach((movie) => {
            const li = document.createElement("li");
            const movieName = movie["Movie name"];
            li.textContent = movieName;
            li.classList.add("movie-list-item"); // Add a
// class to the list item

            li.addEventListener("click", () => {
                getMovieDetails(movieName);
            });
            ul.appendChild(li);
        });
        movieCategoryDiv.appendChild(ul);
        movieCategoryDiv.style.display = "block";
        // Show the box when movies are available
    }
}

function displayMovieDetails1(details, movieName) {
    const movieDetailsDiv =
document.getElementById("movie-details1");
    movieDetailsDiv.innerHTML = "";

    if (details) {
        const description = details["Description"];
        const actors = details["Actors"];
        const detailsText = `
            <div><strong>Movie Name:</strong>
${movieName}</div>
            <div><strong>Description:</strong>
${description}</div>

```

```

        <div><strong>Actors:</strong> ${actors}</div>
    `;
    movieDetailsDiv.innerHTML = detailsText;

    movieDetailsDiv.style.display = "block";
    // Show the box when details are available
    }
}
// Call the function to fetch movies when
// the page loads
document.addEventListener("DOMContentLoaded",
getMovieList());
</script>
</body>
</html>

```

The book-seats-form.html form:

- Uses HTML and JavaScript.
- Defines three buttons to filter the movie names by category and defines the `onclick` event for these buttons **Action**, **Comedy**, and **Drama**.
- Displays movie names in a list using the `movie-category1` division.
- Displays movie details using the `movie-details1` division.
- Defines the `getMovieList` function that fetches the movie names from the MongoDB server using `/get-all-movies` endpoint. This function calls the `displayMovieList` function to list the movie names on the page.
- Defines the `getMovies` function that fetches the movie based on the specific category using `/get-movies` endpoint. This function calls the `displayMovies` function to show the list of movies in a specific category.
- Defines the `getMovieDetails` function that fetches details of a selected movie using `/get-movie-details` endpoint. This function calls the `displayMovieDetails1` function to display the details of the selected movie.

- Defines the `displayMovies` function displays a list of movies in a specified container. This function:
 - Checks if there are any movies available for the specific category.
 - Creates an unordered list (`ul`) by iterating through the list of movies for the specified category. For each movie it creates a list item (`li`) and sets the text of the `li` to the movie name. The function then, attaches an event listener to the list item. When a list item is clicked, the function calls the `getMovieDetails` function with the selected movie name. Each list item is appended to the unordered list.
 - Appends the unordered list to the movie category container.

Save the code with the file name `book-seats-form.html` in the `Templates` folder.

➤ `movie-list.js`

Finally, create the main Node.js application file, `movie-list.js`. This file acts as the main entry point for the movie application. In this file, the core logic of the application is handled. Routes and middleware are also defined.

Consider the code in Code Snippet 5 for `movie-list.js` file.

Code Snippet 5:

```
const express = require("express");
const { MongoClient } = require("mongodb");
const bodyParser = require("body-parser");
const session = require("express-session");
const passport = require("passport");
const LocalStrategy = require("passport-local");
const path = require("path"); // Import the path module
// const uri =
//
// "mongodb+srv://<<username>>:<<password>>@moviecollectioncluster.4x9rauo.mongodb.net/moviecollectioncluster?retryWrites=true&w=majority"; //MongoDB Atlas Link
//Connect to MongoDB using MongoDB Compass localhost
const uri = "mongodb://127.0.0.1:27017/Movie";
// MongoDB connection URI
const client = new MongoClient(uri);
const app = express();

app.use(bodyParser.urlencoded({ extended: false }));

async function main() {
```

```

try {
  await client.connect();
  console.log("Connected to MongoDB");

  const database = client.db();
  const collection =
database.collection("MovieCollection");
  // Adjust the collection name

  // Set up the views folder
  app.set("views", path.join(__dirname, "views"));

  app.get("/", (req, res) => {
    console.log("Received request for /");
    // Serve the HTML page for displaying movie lists
    res.sendFile(__dirname + "/Templates/movie-list.html");
  });

  // Serve Add_movie-form.html
  app.get("/add-movie-form.html", (req, res) => {
    console.log("Received request for /add-movie-
form.html");
    // Serve the HTML form
    res.sendFile(__dirname + "/Templates/add-movie-
form.html");
  });

  // Serve book-seats-form.html
  app.get("/book-seats-form.html", (req, res) => {
    res.sendFile(__dirname + "/Templates/book-seats-
form.html");
  });

  app.get("/get-movies", async (req, res) => {
    const category = req.query.category;
    // Get the selected category from query parameters

    try {
      // Fetch movies of the selected category
      // from the database
      const movies = await collection.find({ Category:
category }).toArray();
      res.status(200).json(movies);
    } catch (error) {
      console.error("Error fetching movies:", error);
      res.status(500).json({ error: "Failed to fetch
movies" });
    }
  });

  app.get("/get-all-movies", async (req, res) => {

```

```

        try {
            // Fetch movies of the selected category from the
database
            const movies = await collection.find().toArray();
            res.status(200).json(movies);
        } catch (error) {
            console.error("Error fetching movies:", error);
            res.status(500).json({ error: "Failed to fetch
movies" });
        }
    });

    app.get("/get-movie-details", async (req, res) => {
        const movieName = req.query.name; // Get the selected
// movie name from query parameters

        try {
            // Fetch movie details based on the selected
// movie name from the database
            const movie = await collection.findOne({ "Movie
name": movieName });
            if (movie) {
                res.status(200).json({
                    Description: movie["Description"],
                    Actors: movie["Actors"],
                });
            } else {
                res.status(404).json({ error: "Movie not found" });
            }
        } catch (error) {
            console.error("Error fetching movie details:",
error);
            res.status(500).json({ error: "Failed to fetch movie
details" });
        }
    });

    // Handle the form submission and add a movie
    app.post("/add-movie", async (req, res) => {
        try {
            // Insert a new movie document into the MongoDB
collection
            await collection.insertOne(req.body);
            console.log("Added movie to the database");
            // Display an alert on the same page
            res.send(
                '<script>alert("Movie added successfully");
window.location.href = "/";</script>'
            );
        } catch (error) {
            console.error("Error adding movie:", error);

```

```

        // Render an error message on the same page
        res.status(500).send("<h2>Failed to add the
movie</h2>");
    }
});
// Handle booking seats
app.post("/book-seats", async (req, res) => {
    try {
        // Retrieve movie information from the request
        const movieNameToBook = req.body["Movie name"];
        const seatsToBook = parseInt(req.body["seats-to-
book"]);

        // Check if the movie exists
        const existingMovie = await collection.findOne({
            "Movie name": movieNameToBook,
        });

        if (!existingMovie) {
            console.log("Movie not found in the database.");
            return res.send("Movie not found in the
database.");
        }

        // Retrieve the available seats from the existing
movie document
        const availableSeats = existingMovie["Available
Seats"]; //Here field name should be same with databse field
name

        if (seatsToBook <= availableSeats) {
            // Calculate the updated available seats
            const updatedAvailableSeats = availableSeats -
seatsToBook;

            // Update the document with new available seats
            const result = await collection.updateOne(
                { "Movie name": movieNameToBook },
                { $set: { "Available Seats":
updatedAvailableSeats } } //Here field name should be same
with databse field name
            );

            if (result.modifiedCount === 1) {
                // Display an alert message on the same page
                const alertMessage = `Booking successful for
${seatsToBook} seat(s) in ${movieNameToBook}`;
                return res.send(`
<script>
                    alert("${alertMessage}");
                    window.location.href = "${"/}";

```

```

        </script>
    `);
    } else {
        console.log("Failed to update available seats");
        const errorMessage = "Failed to update available
seats";

        // Display an alert on the same page
        return res.send(`
<script>
    alert("${errorMessage}");
    window.location.href = "${"/}";
</script>
`);
    }
    } else {
        console.log(
            `Not enough seats available for ${seatsToBook}
seat(s) in ${movieNameToBook}`
        );
        return res.send(
            `Not enough seats available for ${seatsToBook}
seat(s) in ${movieNameToBook}` // This will display message
        );
    }
    } catch (error) {
        console.error("Error booking seats:", error);
        return res.status(500).send("Failed to book seats");
    }
    });
} catch (error) {
    console.error("Error connecting to MongoDB:", error);
}
}

main().catch(console.error);

app.listen(process.env.PORT || 3000, process.env.IP ||
"0.0.0.0", () => {
    console.log("Server is running");
});

```

The movie-list.js file:

- **Imports packages:** Various Node.js packages such as express, mongoose, passport, body-parser, passport-local, and passport-local-mongoose are imported.
- **Creates express application:** The statement `const app = express()` creates the express application.

- **Connects to the database:** The statement (mongodb+srv://prachigupta8585:prachi8585@moviecollectioncluster.4x9rauo.mongodb.net/moviecollectioncluster?retryWrites=true&w=majority) connects the application to the MongoDB database, Library, using the connection string.
- **Sets up the middleware:** Packages such as body-parser, express-session, and passport-authenticate are used to set up the middleware.
- **Defines routes:** The application defines several routes for handling Hypertext Transfer Protocol (HTTP) requests. The routes are:
 - /: This is the home page route.
 - /get-movies: This is to fetch movies of the selected category from the database.
 - /get-all-movies: This is to fetch all movie names from the database.
 - /get-movie-details: This is to fetch movie details based on the selected movie name from the database.
 - /add-movie: This is to handle form submission and add a new movie.
 - /book-seats: This is to handle booking seats.
- **Listens for requests:** The application listens on the specified port, either from the environment variable or port 3000. A message is logged when the server starts.

Save the code with the file name `movie-list.js` in the `movie-application` folder.

11.5 Running the Application

To view the application in the browser:

1. Open Command Prompt.
2. Navigate to the movie application folder.
3. Execute the main application file using the command:

```
node movie-list.js
```

Figure 11.10 shows the output of this command.

```
C:\Node\Movie Application>node movie-list.js
Server is running
Connected to MongoDB
```

Figure 11.10: Application in the Browser

4. Open a browser window.
5. Type the URL:

```
localhost:3000
```

Since the Web application is not fully developed yet, the page only displays the output as shown in Figure 11.11.

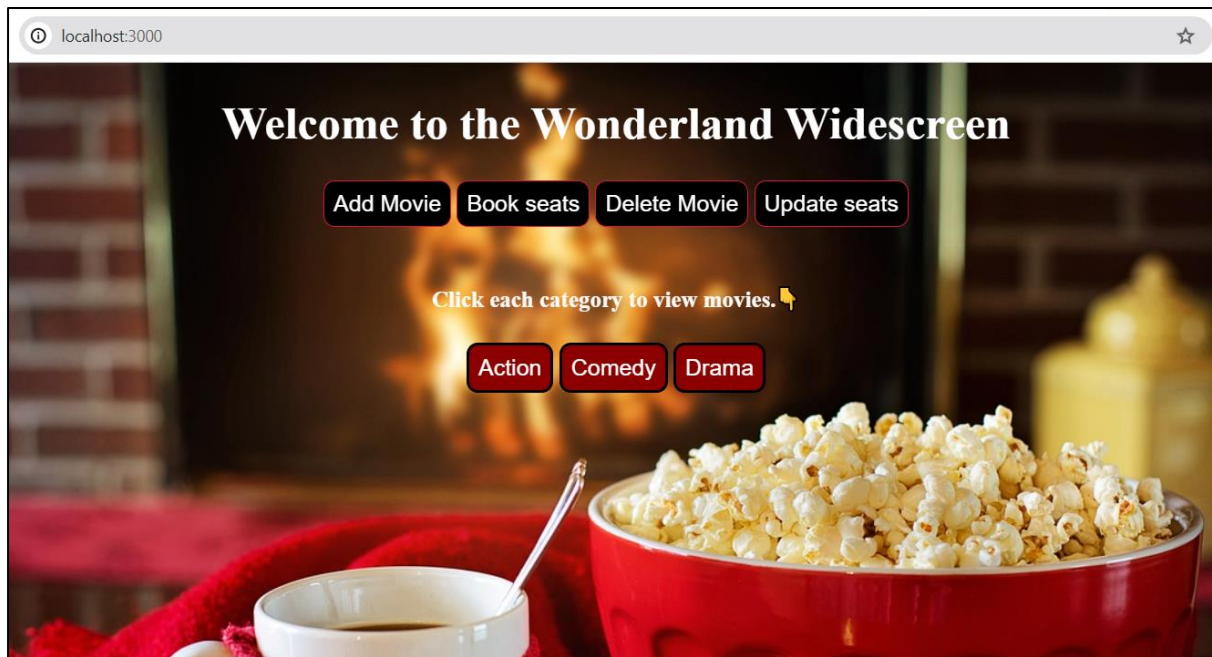


Figure 11.11: Application in the Browser

The application is not yet fully complete hence, further steps to add more pages and deploy the application are covered in the next session.

11.6 Summary

- Creation of a complete Web application involves a combination of several different client-side and server-side technologies.
- The connection string created in the MongoDB cloud database connects the Wonderland Widescreen application to the MongoDB cloud.
- Front-end files are created using HTML and JavaScript.
- The main application entry point is a JavaScript file that must be kept in the root folder of the application.
- HTML files are kept in the `Templates` folder under the root folder.
- When Node packages are installed, they are automatically stored in the `node_modules` folder under the root folder.

Test Your Knowledge

1. Which of the following files will you edit if an error is encountered while deploying the Node.js application on Render?

- a) package-lock.json
- b) package.json
- c) app.js
- d) render.json

2. Consider the code given here.

```
const express = require('express');
const app = express();
const ejs = require('ejs');
app.set('view engine' , 'ejs');
app.get('/', (req , res) => {
  const data = {
    Employee_designation: 'Analyst',
    Employee_ID: 'SP123',
    Company_Location: 'Ontario'
  };
  res.render('index', data) ;
});
app.listen(3000, () => {
  console.log(' Server started on port 3000 ');
});
```

Which of the following code snippets will you use to display Employee_Designation and Employee_Location in the index.html file?

- a) `<body>`
 `<h1> <%= Employee_designation %> </h1>`
 `<h2> <%= Company_Location %> </h3>`
 `</body>`
- b) `<body>`
 `<h1> <%= Employee %> </h1>`
 `<h2> <%= Location %> </h3>`
 `</body>`
- c) `<body>`
 `<h1> <%= Employee_designation %> </h1>`
 `</body>`
- d) `<body>`
 `<h1> <%= designation %> </h1>`
 `<h2> <%= Location %> </h3>`
 `</body>`

3. Which of the following middleware applications helps you add admin and user login to your Web application?
- a) passport-local
 - b) local
 - c) authentication
 - d) ejs
4. You are working on user authentication for your Web application. If incorrect credentials are entered for the admin login, the application must display a message on a pop-up. Which of the following will you use to achieve this?
- a) `<script>alert("Incorrect Admin username or password")</script>`
 - b) `alert("Incorrect Admin username or password")`
 - c) `res.send("Incorrect Admin username or password")`
 - d) `res.send('<script>alert("Incorrect Admin username or password"); window.location.href = "/";</script>');`
5. Which function will you use to store the user's ID, username, or any other information?
- a) `passport.use`
 - b) `serializeUser` and `deserializeUser`
 - c) `passport`
 - d) `serialize`

Answers to Test Your Knowledge

1	b
2	a
3	a
4	d
5	b

Try It Yourself

1. Create a simple Web application for the **Student Management System**. Perform the given operations.

- a) Create an application directory.
- b) Import the necessary packages.
- c) Create a database named Student. In this database add the `Student_Collection` collection and insert 3 documents as given in Table 11.1.

Name	Age	Marks
Alex	21	93
John	20	80
Harry	22	79

Table 11.1: Student_Collection Data

- d) Create an HTML form to view all the information in the inserted documents in a list format.
- e) Create an HTML form to add a new student to the collection. Use input text boxes to accept the details of the new student. Add a **Submit** button to add the information to the MongoDB cloud database.
- f) Create an HTML form to update student information. In this page, add a listbox to list the names of all the students. When a student is selected in the listbox, age and marks information of the student must be fetched in two fields. The user can edit the data in these two fields. Add a **Submit** button to update the database with the edited information in the fields.
- g) Create a home page for the application to contain two buttons **Add** and **Update**. When these buttons are clicked the application must redirect the user to the corresponding forms.
- h) Update the `package.json` file to include the main application name. Also, include the `engines` and `node` fields.